COVID-19 Diagnosis Prediction Under Noisy Labels

Fixing the Data Before Fixing the Model

Cyrus Chung Hussain Elahi Kayin Malcolm Sohail (Neel) Sarkar

STA314, University of Toronto

March 2026

Team **Unemployed_Statisticians** | Final Rank: **#2** | Private Score: **0.99130**

The Task and the Twist

Binary classification: predict whether a patient tested positive for COVID-19 from symptoms, demographics, and clinical indicators.

Dataset at a glance

- ∼1,500 training rows, ∼3,500 test rows
- 18 features: 3 continuous, 2 categorical, 13 binary
- Balanced classes (778 pos. / 722 neg.)
- `comorbidity` 53% NaN, recoded as "None"

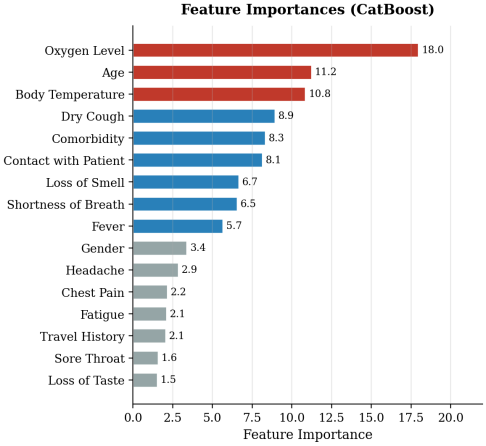
The catch

- The instructor *deliberately flipped* a subset of training labels
- Roughly **19% of labels are wrong**
- Models trained on noisy labels plateaued at 74% to 80%
- Scored on **accuracy**: you get the label right or you do not

Label noise shaped our entire approach.

Data Overview and Feature Importance

- `oxygen_level` dominates (importance ≈ 18)
- `age` and `body_temperature` are next
- Key symptoms: `dry_cough`, `shortness_of_breath`, `loss_of_smell`
- Many binary features contribute $<$ 5% each but collectively matter



CatBoost feature importances (exploratory).

Why Standard Models Fail Here

The core issue: with ∼19% label noise, any model that fits training labels hard will learn the noise.

Model	CV Accuracy	Problem
CatBoost (Optuna-tuned)	0.7487	Memorises noisy labels
LightGBM	0.7673	Same issue
LDA	0.7567	Linear, underpowered
QDA	0.7167	High variance, small n
GaussianNB	0.7013	Assumes feature independence
k-NN (k=31)	0.7047	Noise in neighbours
Logistic Regression (T1+T2)	0.9147	Decent after label correction
TabPFN 5-seed (T1+T2)	**0.9460**	**Best by far**

- All tree-based and classical models hit 0.70 to 0.77 on *noisy* labels. No tuning broke through.
- Even LR jumps to 0.91 once labels are fixed. The bottleneck was **label quality**, not model complexity.

What is TabPFN?

TabPFN (Hollmann et al., 2022) is a transformer *meta-trained* on millions of synthetic tabular classification datasets.

How it works

- The entire training set is fed as *context* in a single forward pass, similar to prompting an LLM
- Predictions via **in-context learning**: no gradient updates, no fine-tuning on your data
- Approximates the Bayesian posterior predictive: $P(y_{\text{test}} \mid X_{\text{test}}, X_{\text{train}}, y_{\text{train}})$

Why it fits our task

1. **Pre-training = regularisation.** Cannot overfit to 1,500 rows since it never trains on them via gradient descent
2. **Noise robustness.** Bayesian posterior smooths over individual flipped labels
3. **No hyperparameters.** Zero risk of overfitting to CV folds

Binary classification is the natural sweet spot for TabPFN. The prior was meta-trained heavily on 2-class tasks, and the output space (a single probability) is simple and well-calibrated.

Noise Detection: Three Independent Methods

Instead of relying on any single noise detection technique, we ran three methods that use *completely different* signals and combined their outputs.

Method 1: Confident Learning (Cleanlab).
5-fold OOF logistic regression $\rightarrow$ Cleanlab estimates joint distribution of noisy vs. true labels. Flagged **285** samples (19.0%).

Method 2: Area Under the Margin (AUM, CatBoost).
Compute margin $m_i(t) = P_t(\text{true class}) - P_t(\text{other class})$ at 5 CatBoost checkpoints (100 to 500 iters).
Samples with mean AUM < 0.40 are consistently hard-to-learn. Flagged **352** samples.

Method 3: Dataset Cartography.
5 seeds $\times$ 5 folds of LR $\rightarrow$ mean per-sample confidence $P(\text{true label})$.
Hard-to-learn threshold: mean confidence < 0.45. Flagged **302** samples.

One method is probabilistic, one is margin-based, one is training-dynamics-based.
What matters is where they agree.

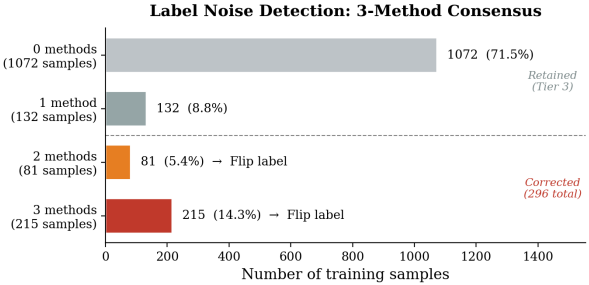
Consensus Tiers and Label Correction

Voting rule: count how many of 3 methods flagged each sample.

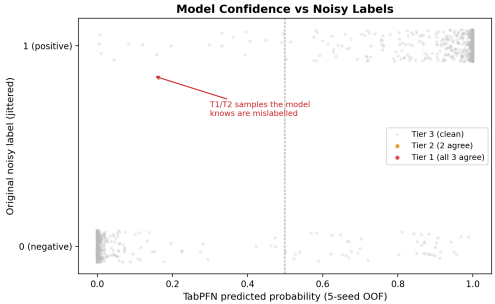
- **Tier 1** (all 3 agree): **215** samples
  → Label flipped. Near-certain noise.
- **Tier 2** (2 of 3 agree): **81** samples
  → Label flipped. Probable noise.
- **Tier 3** (0 or 1 method): **1,204** samples
  → Label retained as given.

Total corrected: 296 labels.

This is a precision/recall tradeoff on noise detection itself. We chose precision: flipping a correct label is just as damaging as keeping a wrong one.



Impact: Before vs. After Label Correction



TabPFN OOF probabilities vs. original labels. Clusters in the lower-right and upper-left are the corrected samples.

Progression of private scores:

Strategy	Private
S1: T1 only	0.99015
S2: +Features	0.98261 ↓
S3: T1+T2	0.99073
S4: 5-seed	**0.99130**

- S1→S3: Tier 2 corrections give **+0.00058**
- S2: feature engineering *hurt* (−0.00754)
- S3→S4: 5-seed ensemble adds **+0.00057**
- 10 total submissions, no leaderboard probing

The 5-Seed Ensemble: Variance Reduction

Problem: TabPFN has stochastic internal processing. Different random seeds produce slightly different probability estimates.

Solution: Train with 5 seeds (42, 123, 314, 777, 2026), average the probability vectors.

Pipeline per seed:

1. 5-fold stratified CV $\rightarrow$ OOF probabilities
2. Full-train model $\rightarrow$ test probabilities
3. Repeat for all 5 seeds

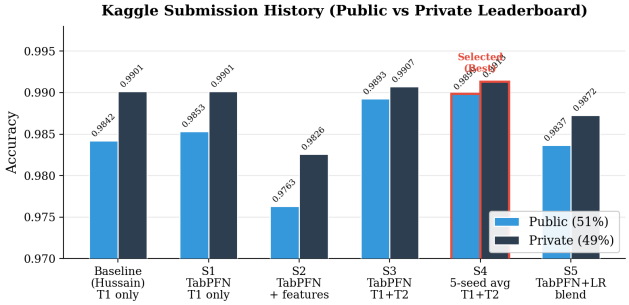
$\Rightarrow$ 25 OOF estimates per training sample
$\Rightarrow$ 5 test prediction vectors, averaged

Why this works:

- This is *not* diversity-based ensembling (blending different model types)
- It is pure **variance reduction** over the same architecture
- Blending TabPFN with LR actually *hurt* (0.9220 vs. 0.9460). Diluting a strong model with a weaker one reduces signal.

Final prediction: classify at threshold 0.50. Threshold sweeping on OOF showed no meaningful gain.

Competition Trajectory



- Only **10 total submissions**; deliberate, not leaderboard-probing
- Each submission tested exactly one hypothesis
- Private (0.99130) $>$ Public (0.98985)
  $\rightarrow$ **no overfitting to public split**
- S2 (feature eng.) was the only regression, confirming TabPFN's prior $>$ hand-crafted features
- Final rank: **#2 overall**

Key Takeaways

1. **Label quality matters more than model complexity.**
   Every model trained on noisy labels hit a ceiling of 74% to 80%. After cleaning 296 labels, we hit 99%+.
2. **Consensus-based noise detection is safer than any single method.**
   Three independent signals agreeing on the same sample is strong evidence. Voting filters false positives.
3. **Pre-trained transformers can work on tabular data, under the right conditions.**
   Small n, binary classification, clean(ed) labels. In-context learning avoids overfitting.
4. **Feature engineering can hurt when the model's prior is already stronger.**
   On 1,500 rows, TabPFN's inductive bias outperformed manual interactions.
5. **Ensemble the same strong model; do not blend with weaker ones.**
   Multi-seed averaging reduces variance. Blending with LR diluted the signal.

Conclusion

Private Score: 0.99130 | Rank: #2

What we did

- 3-method noise detection pipeline
- 296 label corrections (215 T1 + 81 T2)
- TabPFN v2, 5-seed ensemble
- No feature engineering, no threshold tuning
- 10 principled submissions

What mattered most

- The jump from $\sim$0.75 CV accuracy to 0.99 on Kaggle came from **fixing labels**
- Clean data did the heavy lifting, not model choice
- Private $>$ Public confirms no overfitting

Future work

- Pseudo-labelling of confident test predictions
- Extending correction into Tier 3

Code: `https://github.com/n33levo/unemployed_statistician`